Solution Code - Module 3-Core Java - ExecutorServiceCallable

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: Q41_ExecutorServiceCallable.java

```
 1 | import java.util.ArrayList;
 2 | import java.util.List;
 3 | import java.util.concurrent.Callable;
 4 | import java.util.concurrent.ExecutorService;
 5 | import java.util.concurrent.Executors;
 6 | import java.util.concurrent.Future;
 7 |
 8 | // Callable task that calculates the square of a number and simulates work
 9 | class SquareCalculator implements Callable<Integer> {
10 |     private final int number;
11 |
12 |     public SquareCalculator(int number) {
13 |         this.number = number;
14 |     }
15 |
16 |     @Override
17 |     public Integer call() throws Exception {
18 |         // Simulate a small delay (e.g., 200 milliseconds)
19 |         Thread.sleep(200);
20 |         System.out.println(Thread.currentThread().getName() + " calculated square of " + number);
21 |         return number * number;
22 |     }
23 | }
24 |
25 | public class Q41_ExecutorServiceCallable {
26 |     public static void main(String[] args) {
27 |         System.out.println("=== ExecutorService and Callable Concurrency Demo ===");
28 |
29 |         // Create a fixed thread pool with 3 threads
30 |         int threadPoolSize = 3;
31 |         ExecutorService executor = Executors.newFixedThreadPool(threadPoolSize);
32 |         System.out.println("Thread pool initialized with " + threadPoolSize + " threads.");
33 |
34 |         // List to hold Future objects
35 |         List<Future<Integer>> futureList = new ArrayList<>();
36 |
37 |         // Submit 5 tasks to the executor
38 |         System.out.println("\nSubmitting 5 tasks to calculate squares of numbers 1 through 5...");
39 |         for (int i = 1; i <= 5; i++) {
40 |             Callable<Integer> task = new SquareCalculator(i);
41 |             Future<Integer> future = executor.submit(task);
42 |             futureList.add(future);
43 |         }
```

Solution Code - Module 3-Core Java - ExecutorServiceCallable

```
44 |
45 |         // Collect results using Future.get()
46 |         System.out.println("\nRetrieving results from Futures (blocking operations)...");
47 |         int sumOfSquares = 0;
48 |         try {
49 |             for (int i = 0; i < futureList.size(); i++) {
50 |                 Future<Integer> future = futureList.get(i);
51 |                 // Future.get() blocks until the specific task finishes and returns result
52 |                 int result = future.get();
53 |                 System.out.println("Task " + (i + 1) + " result: " + result);
54 |                 sumOfSquares += result;
55 |             }
56 |             System.out.println("\nSum of all squares: " + sumOfSquares);
57 |         } catch (Exception e) {
58 |             System.err.println("An error occurred during thread execution: " + e.getMessage());
59 |             e.printStackTrace();
60 |         } finally {
61 |             // Shut down the executor service
62 |             System.out.println("\nShutting down the executor service...");
63 |             executor.shutdown();
64 |             System.out.println("Executor service shut down.");
65 |         }
66 |     }
67 | }
```